Most HackerRank challenges require you to read input from stdin (standard input) and write output to stdout (standard output).

One popular way to read input from stdin is by using the Scanner class and specifying the Input Stream as System.in. For example:

```
Scanner scanner = new Scanner(System.in);
String myString = scanner.next();
int myInt = scanner.nextInt();
scanner.close();

System.out.println("myString is: " + myString);
System.out.println("myInt is: " + myInt);
```

The code above creates a Scanner object named ***scanner*** and uses it to read a String and an int. It then closes the Scanner object because there is no more input to read, and prints to stdout using System.out.println(String). So, if our input is:

```
Hi 5
```

Our code will print:

```
myString is: Hi
myInt is: 5
```

Change Theme Language Java 7

```java
import java.util.*;

public class Solution {

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        int a = scan.nextInt();
        int b = scan.nextInt();
        int c = scan.nextInt();

        scan.close();

        System.out.println(a);
        System.out.println(b);
        System.out.println(c);

    }

}
```

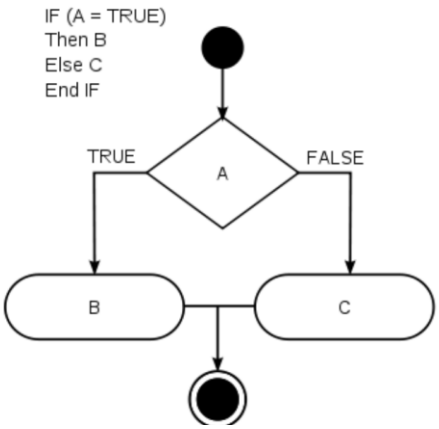
Line: 20 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

1.

2.

In this challenge, we test your knowledge of using if-else conditional statements to automate decision-making processes. An if-else statement has the following logical flow:



Change Theme Language Java 7

```java
import java.util.Scanner;

public class Solution {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int N = sc.nextInt();

        if (N % 2 != 0) {
            System.out.println("Weird");
        } else {
            if (N >= 2 && N <= 5) {
                System.out.println("Not Weird");
            } else if (N >= 6 && N <= 20) {
                System.out.println("Weird");
            } else {
                System.out.println("Not Weird");
            }
        }

        sc.close();
    }
}
```

Line: 23 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

3.

In this challenge, you must read an integer, a double, and a String from stdin, then print the values according to the instructions in the Output Format section below. To make the problem a little easier, a portion of the code is provided for you in the editor.

Note: We recommend completing Java Stdin and Stdout I before attempting this challenge.

Input Format

There are three lines of input:

1. The first line contains an integer.
2. The second line contains a double.
3. The third line contains a String.

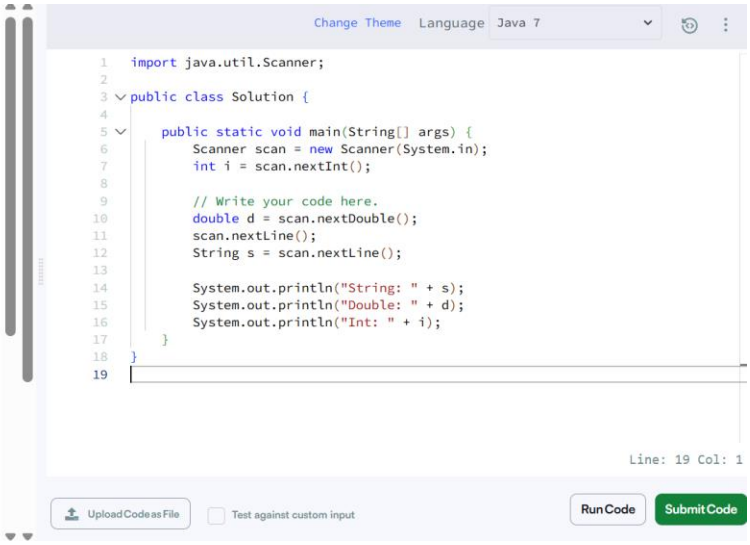
Output Format

There are three lines of output:

1. On the first line, print `String:` followed by the unaltered String read from stdin.
2. On the second line, print `Double:` followed by the unaltered double read from stdin.
3. On the third line, print `Int:` followed by the unaltered integer read from stdin.

To make the problem easier, a portion of the code is already provided in the editor.

Note: If you use the nextLine() method immediately following the nextInt() method, recall that nextInt() reads integer tokens; because of this, the last newline character for that line of integer input is still queued in the input buffer and the next nextLine() will be



4.

Java's System.out.printf function can be used to print formatted output. The purpose of this exercise is to test your understanding of formatting output using printf.

To get you started, a portion of the solution is provided for you in the editor; you must format and print the input to complete the solution.

Input Format

Every line of input will contain a String followed by an integer.
Each String will have a maximum of 10 alphabetic characters, and each integer will be in the inclusive range from 0 to 999.

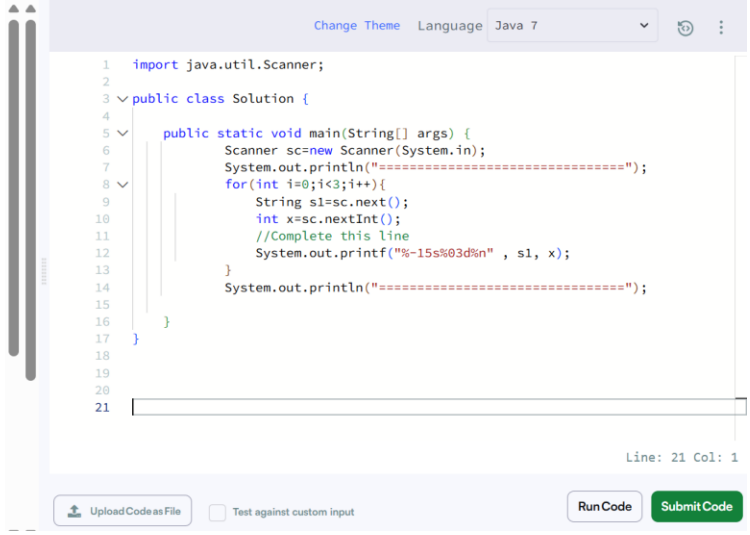
Output Format

In each line of output there should be two columns:
The first column contains the String and is left justified using exactly 15 characters.
The second column contains the integer, expressed in exactly 3 digits; if the original input has less than three digits, you must pad your output's leading digits with zeroes.

Sample Input

```
java 100
cpp 65
python 50
```

Sample Output



5.

Objective

In this challenge, we're going to use loops to help us do some simple math.

Task

Given an integer, N, print its first 10 multiples. Each multiple $N \times i$ (where $1 \leq i \leq 10$) should be printed on a new line in the form: `N x i = result`.

Input Format

A single integer, N.

Constraints

- $2 \leq N \leq 20$

Output Format

Print 10 lines of output; each line i (where $1 \leq i \leq 10$) contains the *result* of $N \times i$ in the form:
`N x i = result`.

Sample Input

```
2
```

Sample Output

Change Theme Language Java 7

```java
import java.util.Scanner;

public class Solution {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int N = sc.nextInt();

        for (int i = 1; i <= 10; i++) {
            System.out.println(N + " x " + i + " = " + (N * i));
        }

        sc.close();
    }
}
```

Line: 15 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

6.

We use the integers a, b, and n to create the following series:

$$(a + 2^0 \cdot b), (a + 2^0 \cdot b + 2^1 \cdot b), \ldots, (a + 2^0 \cdot b + 2^1 \cdot b + \ldots + 2^{n-1} \cdot b)$$

You are given q queries in the form of a, b, and n. For each query, print the series corresponding to the given a, b, and n values as a single line of n space-separated integers.

Input Format

The first line contains an integer, q, denoting the number of queries.
Each line i of the q subsequent lines contains three space-separated integers describing the respective a_i, b_i, and n_i values for that query.

Constraints

- $0 \leq q \leq 500$
- $0 \leq a, b \leq 50$
- $1 \leq n \leq 15$

Output Format

For each query, print the corresponding series on a new line. Each series must be printed in order as a single line of n space-separated integers.

Sample Input

Change Theme Language Java 7

```java
import java.util.Scanner;

public class Solution {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int q = sc.nextInt();

        while (q-- > 0) {
            int a = sc.nextInt();
            int b = sc.nextInt();
            int n = sc.nextInt();

            int sum = a;

            for (int i = 0; i < n; i++) {
                sum += (1 << i) * b;
                System.out.print(sum);

                if (i != n - 1) {
                    System.out.print(" ");
                }
```

7.

Java has 8 primitive data types; char, boolean, byte, short, int, long, float, and double. For this exercise, we'll work with the primitives used to hold integer values (byte, short, int, and long):

- A byte is an 8-bit signed integer.
- A short is a 16-bit signed integer.
- An int is a 32-bit signed integer.
- A long is a 64-bit signed integer.

Given an input integer, you must determine which primitive data types are capable of properly storing that input.

To get you started, a portion of the solution is provided for you in the editor.

Reference: https://docs.oracle.com/javase/tutorial/java/nutsandbolts/datatypes.html

Input Format

The first line contains an integer, T, denoting the number of test cases.
Each test case, T, is comprised of a single line with an integer, n, which can be arbitrarily large or small.

Output Format

For each input variable n and appropriate primitive $dataType$, you must determine if the given primitives are capable of storing it. If yes, then print:

```
n can be fitted in:
```

Change Theme Language Java 7

```java
import java.util.*;

class Solution {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int t = sc.nextInt();

        for (int i = 0; i < t; i++) {
            try {
                long x = sc.nextLong();
                System.out.println(x + " can be fitted in:");

                if (x >= Byte.MIN_VALUE && x <= Byte.MAX_VALUE)
                    System.out.println("* byte");

                if (x >= Short.MIN_VALUE && x <= Short.MAX_VALUE)
                    System.out.println("* short");

                if (x >= Integer.MIN_VALUE && x <= Integer.MAX_VALUE)
                    System.out.println("* int");
```

Line: 33 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

8.

> "In computing, End Of File (commonly abbreviated EOF) is a condition in a computer operating system where no more data can be read from a data source." — (Wikipedia: End-of-file)

The challenge here is to read n lines of input until you reach EOF, then number and print all n lines of content.

Hint: Java's Scanner.hasNext() method is helpful for this problem.

Input Format

Read some unknown n lines of input from stdin(System.in) until you reach EOF; each line of input contains a non-empty String.

Output Format

For each line, print the line number, followed by a single space, and then the line content received as input.

Sample Input

```
Hello world
I am a file
Read me until end-of-file.
```

Sample Output

Change Theme Language Java 7

```java
import java.util.Scanner;

public class Solution {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int lineNumber = 1;

        while (sc.hasNextLine()) {
            String line = sc.nextLine();
            System.out.println(lineNumber + " " + line);
            lineNumber++;
        }

        sc.close();
    }
}
```

Line: 17 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

9.

"A string is traditionally a sequence of characters, either as a literal constant or as some kind of variable." — Wikipedia: String (computer science)

This exercise is to test your understanding of Java Strings. A sample String declaration:

```
String myString = "Hello World!"
```

The elements of a String are called characters. The number of characters in a String is called the length, and it can be retrieved with the String.length() method.

Given two strings of lowercase English letters, A and B, perform the following operations:

1. Sum the lengths of A and B.
2. Determine if A is lexicographically larger than B (i.e.: does B come before A in the dictionary?).
3. Capitalize the first letter in A and B and print them on a single line, separated by a space.

Input Format

The first line contains a string A. The second line contains another string B. The strings are comprised of only lowercase English letters.

Output Format

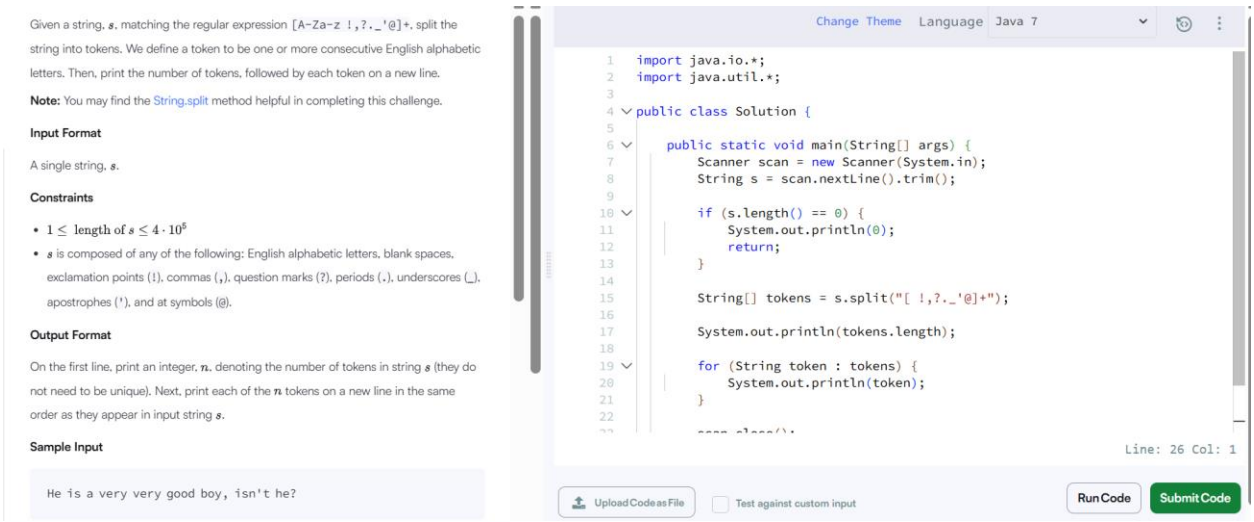
Change Theme Language Java 7

```java
import java.util.Scanner;

public class Solution {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String A = sc.next();
        String B = sc.next();

        // Sum of lengths
        System.out.println(A.length() + B.length());

        // Lexicographical comparison
        if (A.compareTo(B) > 0)
            System.out.println("Yes");
        else
            System.out.println("No");

        // Capitalize first letter
        A = A.substring(0, 1).toUpperCase() + A.substring(1);
        B = B.substring(0, 1).toUpperCase() + B.substring(1);
```

Line: 29 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

10.



Given a string, s, matching the regular expression `[A-Za-z !,?._'@]+`, split the string into tokens. We define a token to be one or more consecutive English alphabetic letters. Then, print the number of tokens, followed by each token on a new line.

Note: You may find the String.split method helpful in completing this challenge.

Input Format

A single string, s.

Constraints

- $1 \leq$ length of $s \leq 4 \cdot 10^5$
- s is composed of any of the following: English alphabetic letters, blank spaces, exclamation points (!), commas (,), question marks (?), periods (.), underscores (_), apostrophes ('), and at symbols (@).

Output Format

On the first line, print an integer, n, denoting the number of tokens in string s (they do not need to be unique). Next, print each of the n tokens on a new line in the same order as they appear in input string s.

Sample Input

```
He is a very very good boy, isn't he?
```

Change Theme Language Java 7

```java
import java.io.*;
import java.util.*;

public class Solution {

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        String s = scan.nextLine().trim();

        if (s.length() == 0) {
            System.out.println(0);
            return;
        }

        String[] tokens = s.split("[ !,?._'@]+");

        System.out.println(tokens.length);

        for (String token : tokens) {
            System.out.println(token);
        }
```

Line: 26 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

11.

Using **Regex**, we can easily match or search for patterns in a text. Before searching for a pattern, we have to specify one using some well-defined syntax.

In this problem, you are given a pattern. You have to check whether the syntax of the given pattern is valid.

Note: In this problem, a regex is only valid if you can compile it using the Pattern.compile method.

Input Format

The first line of input contains an integer N, denoting the number of test cases. The next N lines contain a string of any printable characters representing the pattern of a regex.

Output Format

For each test case, print `Valid` if the syntax of the given pattern is correct. Otherwise, print `Invalid`. Do not print the quotes.

Sample Input

```
3
([A-Z])(.+)
[AZ[a-z](a-z)
batcatpat(nat
```

Change Theme Language Java 7

```
import java.util.Scanner;
import java.util.regex.Pattern;
import java.util.regex.PatternSyntaxException;

public class Solution {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int testCases = Integer.parseInt(sc.nextLine());

        while (testCases-- > 0) {
            String pattern = sc.nextLine();

            try {
                Pattern.compile(pattern);
                System.out.println("Valid");
            } catch (PatternSyntaxException e) {
                System.out.println("Invalid");
            }
        }

        sc.close();
```

Line: 28 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

12.

In a tag-based language like XML or HTML, contents are enclosed between a start tag and an end tag like `<tag>contents</tag>`. Note that the corresponding end tag starts with a `/`.

Given a string of text in a tag-based language, parse this text and retrieve the contents enclosed within sequences of well-organized tags meeting the following criterion:

1. The name of the start and end tags must be same. The HTML code `<h1>Hello World</h2>` is not valid, because the text starts with an `h1` tag and ends with a non-matching `h2` tag.
2. Tags can be nested, but content between nested tags is considered not valid. For example, in `<h1><a>contents</a>invalid</h1>`, `contents` is valid but `invalid` is not valid.
3. Tags can consist of any printable characters.

Input Format

The first line of input contains a single integer, N (the number of lines).
The N subsequent lines each contain a line of text.

Constraints

- $1 \leq N \leq 100$
- Each line contains a maximum of 10^4 printable characters.
- The total number of characters in all test cases will not exceed 10^6.

Change Theme Language Java 7

```
import java.util.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        int testCases = Integer.parseInt(in.nextLine());

        while (testCases-- > 0) {
            String line = in.nextLine();

            boolean found = false;

            Pattern pattern = Pattern.compile("<([^>]+)>([^<>]+)</\\1>");
            Matcher matcher = pattern.matcher(line);

            while (matcher.find()) {
                System.out.println(matcher.group(2));
                found = true;
            }
```

Line: 31 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

13.

Java's BigDecimal class can handle arbitrary-precision signed decimal numbers. Let's test your knowledge of them!

Given an array, s, of n real number strings, sort them in descending order — but wait, there's more! Each number must be printed in the exact same format as it was read from stdin, meaning that $.1$ is printed as $.1$, and 0.1 is printed as 0.1. If two numbers represent numerically equivalent values (e.g., $.1 \equiv 0.1$), then they must be listed in the same order as they were received as input).

Complete the code in the unlocked section of the editor below. You must rearrange array s's elements according to the instructions above.

Input Format

The first line consists of a single integer, n, denoting the number of integer strings. Each line i of the n subsequent lines contains a real number denoting the value of s_i.

Constraints

- $1 \leq n \leq 200$
- Each s_i has at most 300 digits.

Output Format

Locked stub code in the editor will print the contents of array s to stdout. You are only responsible for reordering the array's elements.

Sample Input

14.

In this problem, you have to add and multiply huge numbers! These numbers are so big that you can't contain them in any ordinary data types like a long integer.

Use the power of Java's BigInteger class and solve this problem.

Input Format

There will be two lines containing two numbers, a and b.

Constraints

a and b are non-negative integers and can have maximum 200 digits.

Output Format

Output two lines. The first line should contain $a + b$, and the second line should contain $a \times b$. Don't print any leading zeros.

Sample Input

```
1234
20
```

Sample Output

```
1254
24680
```

15.

You are given a $6 * 6$ 2D array. An hourglass in an array is a portion shaped like this:

```
a b c
  d
e f g
```

For example, if we create an hourglass using the number 1 within an array full of zeros, it may look like this:

```
1 1 1 0 0 0
0 1 0 0 0 0
1 1 1 0 0 0
0 0 0 0 0 0
0 0 0 0 0 0
0 0 0 0 0 0
```

Actually, there are many hourglasses in the array above. The three leftmost hourglasses are the following:

```
1 1 1   1 1 0   1 0 0
  1       0       0
1 1 1   1 1 0   1 0 0
```

The sum of an hourglass is the sum of all the numbers within it. The sum for the

Change Theme Language Java 7

```java
import java.io.*;
import java.util.*;

public class Solution {

    public static void main(String[] args) throws IOException {

        Scanner sc = new Scanner(System.in);
        int[][] arr = new int[6][6];

        // Read the 6x6 array
        for (int i = 0; i < 6; i++) {
            for (int j = 0; j < 6; j++) {
                arr[i][j] = sc.nextInt();
            }
        }

        int maxSum = Integer.MIN_VALUE;

        // Calculate hourglass sums
        for (int i = 0; i < 4; i++) {
            for (int j = 0; j < 4; j++) {
```

Line: 38 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

16.

Sometimes it's better to use dynamic size arrays. Java's Arraylist can provide you this feature. Try to solve this problem using Arraylist.

You are given n lines. In each line there are zero or more integers. You need to answer a few queries where you need to tell the number located in y^{th} position of x^{th} line.

Take your input from System.in.

Input Format

The first line has an integer n. In each of the next n lines there will be an integer d denoting number of integers on that line and then there will be d space-separated integers. In the next line there will be an integer q denoting number of queries. Each query will consist of two integers x and y.

Constraints

- $1 <= n <= 20000$
- $0 <= d <= 50000$
- $1 <= q <= 1000$
- $1 <= x <= n$

Each number will fit in signed integer.

Total number of integers in n lines will not cross 10^5.

Output Format

In each line, output the number located in y^{th} position of x^{th} line. If there is no such position, just print "ERROR!"

Change Theme Language Java 7

```java
import java.util.ArrayList;
import java.util.Scanner;

public class Solution {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        ArrayList<ArrayList<Integer>> list = new ArrayList<ArrayList<Integer>>()

        // Read the lines of integers
        for (int i = 0; i < n; i++) {
            int d = sc.nextInt();
            ArrayList<Integer> row = new ArrayList<Integer>();

            for (int j = 0; j < d; j++) {
                row.add(sc.nextInt());
            }

            list.add(row);
        }
```

Line: 41 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

17.

Let's play a game on an array! You're standing at index 0 of an n-element array named $game$. From some index i (where $0 \leq i < n$), you can perform one of the following moves:

- Move Backward: If cell $i-1$ exists and contains a 0, you can walk back to cell $i-1$.
- Move Forward:
  - If cell $i+1$ contains a zero, you can walk to cell $i+1$.
  - If cell $i+leap$ contains a zero, you can jump to cell $i+leap$.
  - If you're standing in cell $n-1$ or the value of $i+leap \geq n$, you can walk or jump off the end of the array and win the game.

In other words, you can move from index i to index $i+1$, $i-1$, or $i+leap$ as long as the destination index is a cell containing a 0. If the destination index is greater than $n-1$, you win the game.

Function Description

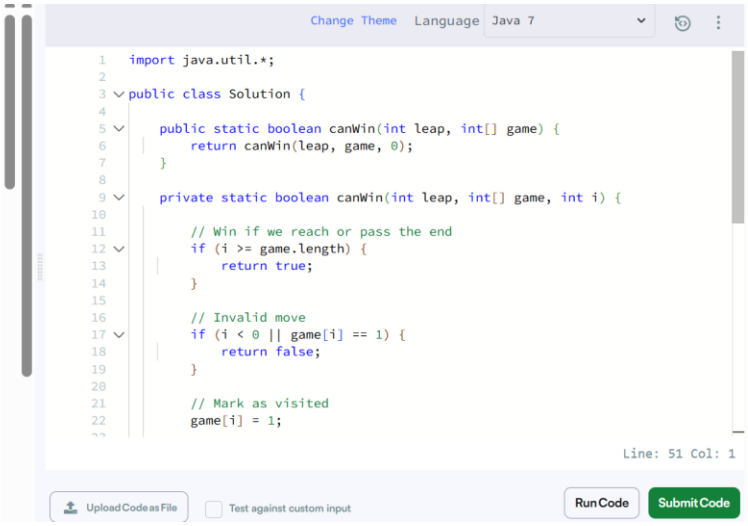
Complete the canWin function in the editor below.

canWin has the following parameters:

- int leap: the size of the leap
- int game[n]: the array to traverse

Returns

- boolean: true if the game can be won, otherwise false



18.

Write the following code in your editor below:

1. A class named Arithmetic with a method named add that takes 2 integers as parameters and returns an integer denoting their sum.
2. A class named Adder that inherits from a superclass named Arithmetic.

Your classes should not be be $\mathbf{public}$.

Input Format

You are not responsible for reading any input from stdin; a locked code stub will test your submission by calling the add method on an Adder object and passing it 2 integer parameters.

Output Format

You are not responsible for printing anything to stdout. Your add method must return the sum of its parameters.

Sample Output

The main method in the Solution class above should print the following:

```
My superclass is: Arithmetic
42 13 20
```